38bece17-ca97-46d7-aa29-39d8f4816407

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'github/gh-aw' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/gh-aw/gh-aw (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-04-05T15:04:15.666Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
feat: mount MCP servers as local CLIs after gateway starts
</pr_title>

<pr_description>
Summary

Adds automatic mounting of MCP servers as local CLI tools in agentic workflows. This behavior is opt-in via a new `tools.mount-as-clis: true` flag (default `false`). When enabled, all user-facing MCP servers — including built-in ones like `github` and `playwright` as well as custom servers — are exposed as standalone shell commands and excluded from the agent's final MCP config so the agent uses the CLI wrappers exclusively. When `false`, no CLI mounting step is emitted and

no prompt section is injected.

How It Works

After the MCP gateway starts, a new "Mount MCP servers as CLIs" workflow step runs `mount_mcp_as_cli.cjs` via `actions/github-script`. This script:

1. Reads a manifest of configured MCP servers (saved by `start_mcp_gateway.sh` before cleanup)
2. For each server, follows the full MCP session protocol (`initialize` → `notifications/initialized` → `tools/list`) to discover available tools
3. Generates a standalone CLI wrapper script at `${RUNNER_TEMP}/gh-aw/mcp-cli/bin/<server-name>`
4. Locks the bin directory (`chmod 555`) so the agent cannot modify or add scripts
5. Adds the directory to `PATH` via `core.addPath()` for subsequent steps

The agent is also informed about these tools via an injected prompt section (wrapped in `<mcp-clis>` XML tags) listing the available CLI commands.

Architecture: Gateway vs. Agent Config

The MCP gateway must start Docker containers for **all** configured servers — including CLI-mounted ones — so they have reachable endpoints for the CLI manifest. The filtering is therefore applied in two stages:

1. **Gateway config** (piped to `start_mcp_gateway.sh`): includes all servers so the gatewnfig` (parsed from `tools.mount-as-clis`, default `false`)

Modified: `pkg/parser/schemas/main_workflow_schema.json`
- Added `mount-as-clis` boolean property to the tools schema

Modified: `pkg/workflow/mcp_rendering.go`
- Removed `composeCLIMountFilter` — CLI-mounted servers are no longer excluded from the gateway config (they must be started by the gateway). Filtering is now delegated to the conversion scripts.

Modified: `pkg/workflow/mcp_setup_generator.go`
- Exports `GH_AW_MCP_CLI_SERVERS` (JSON array built with `json.Marshal`) in the gateway step script, after

`GH_AW_ENGINE`, so conversion scripts can exclude those servers from the agent config

Modified: `pkg/workflow/compiler_yaml_main_job.go`
- Calls `generateMCPCLIMountStep()` after `generateMCPSetup()` completes

Modified: `pkg/workflow/unified_prompt_step.go`
- Injects MCP CLI tools prompt section (wrapped in `<mcp-clis>` XML tags) after safe outputs (section 8a)

Modified: `actions/setup/sh/validate_prompt_placeholders.sh`
- Strips inline backtick code spans before checking for unreplaced `__GH_AW_*__` placeholders, so documented placeholder names in PR descriptions or code examples are not flagged as errors

Modified: `.github/workflows/smoke-copilot.md`
- Enabled `mount-as-clis: true` to exercise the feature in smoke testing
- Added a "Tool Access Overview" section at the top of the test instructions, explicitly listing all CLI-mounted servers and clarifying they must be invoked via bash (not as MCP tools)
- Updated test prompts for tests 1, 3, 4, and 12 to use CLI syntax (e.g. `github pull_request_read --method list ...`, `serena activate_project --path ...`, `playwright browser_navigate --url ...`) with "via bash" guidance to eliminate agent ambiguity

Updated
- All 180 `.lock.yml` files recompiled

Example

For a workflow configured with `mount-as-clis: true`, `playwright`, `github`, and a custom `tavily` MCP server, the agent will have:

```bash
# Shell commands available on PATH (under ${RUNNER_TEMP}/gh-aw/mcp-cli/bin/):
playwright --help
playwright browser_navigate --url https://example.com
github issue_read --method get --owner octocat --repo Hello-World --issue_number 1
tavily search --query "latest AI news"
```

Those servers are **excluded from the agent's MCP config**, so the agent interacts with them exclusively through the CLI wrappers. The gateway still starts all Docker containers so the CLI wrappers can communicate with them.

The prompt injected to the agent (loaded from `mcp_cli_tools_prompt.md`):
```xml
<mcp-clis>
## MCP Servers Mounted as Shell CLI Commands

> **IMPORTANT**: The following MCP servers are **NOT available as MCP tools** in your agent context. They have been mounted exclusively as shell (bash) CLI commands. You **must** call them via the shell — do **not** attempt to use them as MCP protocol tools.

- `github` — run `github --help` to see available tools
- `playwright` — run `playwright --help` to see available tools
- `tavily` — run `tavily --help` to see available tools
...
</mcp-clis>
```

✨ PR Review Safe Output Test - Run 23992552020

> 💥 *THE END] — Illustrated by [Smoke Claude* · ● 232.3K · ◷

✨ PR Review Safe Output Test - Run 23993205568

> 💥 *THE END] — Illustrated by [Smoke Claude* · ● 377K · <a href="https://github.com/search?q=repo%3Agithub%2Fgh-

aw+%22gh-aw-workflow-call-id%3A+github%2Fgh-aw%2Fsmoke-claude%22&type=pullrequests">◷

✨ PR Review Safe Output Test - Run 24001448881

> 💥 *THE END] — Illustrated by [Smoke Claude* · ● 246.1K · ◷

</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

```
<comment_new>
<comment_id>4189018756</comment_id>
<author>@pelikhan</author>
@copilot merge main and recompile
</comment_new>
```

```
</pr_comments>

</comments>

----
The last **22** git commits in this branch are the changes you
have made so far. Use those as your change commit history.
</problem_statement>


Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address the
<comment_new> comments. Reply to new comments as you address them.
Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can
stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and
```

ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prev_assistant>

Done. Merged main (which updated the mcpg image tag from `v0.2.12` to `v0.2.14`) and recompiled all 181 workflows successfully.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'github/gh-aw' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/gh-aw/gh-aw (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-04-06T03:06:05.616Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
feat: mount MCP servers as local CLIs after gateway starts

```
</pr_title>

<pr_description>
## Summary

Adds automatic mounting of MCP servers as local CLI tools in
agentic workflows. This behavior is opt-in via a new
`tools.mount-as-clis: true` flag (default `false`). When
enabled, all user-facing MCP servers — including built-in ones
like `github` and `playwright` as well as custom servers — are
exposed as standalone shell commands and excluded from the
agent's final MCP config so the agent uses the CLI wrappers
exclusively. When `false`, no CLI mounting step is emitted and
no prompt section is injected.

## How It Works

After the MCP gateway starts, a new "Mount MCP servers as CLIs"
workflow step runs `mount_mcp_as_cli.cjs` via `actions/github-
script`. This script:

1. Reads a manifest of configured MCP servers (saved by
`start_mcp_gateway.sh` before cleanup)
2. For each server,...
</pr_description>

## Comments on the PR (you are @copilot in this section)

<comments>

<pr_reviews>

<comment_thread>
<file>.github/workflows/agent-performance-
analyzer.lock.yml:164</file>
<original_commit>33e03db</original_commit>




</comment_thread>

<comment_thread>
<file>actions/setup/sh/mount_mcp_as_cli.sh:1</file>
<original_commit>33e03db</original_commit>
```

````
</comment_thread>

<comment_thread>
<file>.github/workflows/smoke-copilot.lock.yml:232</file>
<original_commit>a6cfa91</original_commit>




</comment_thread>

</pr_reviews>

<pr_comments>



</pr_comments>

<pr_reviews>

<comment_thread>
<file>pkg/workflow/tools_types.go:88-89</file>
<original_commit>d22ea30</original_commit>

nges.
```suggestion
        // a read-only directory added to PATH. The servers
remain in the MCP gateway
        // config, but are filtered out of the agent's final MCP
config so the agent
        // uses the CLI instead of the MCP protocol.
```
</comment_old>

</comment_thread>

<comment_thread>
<file>pkg/parser/schemas/main_workflow_schema.json:4292</file>
<original_commit>d22ea30</original_commit>

<comment_old>
````

<author>@copilot-pull-request-reviewer</author>
Schema description for tools.mount-as-clis claims CLI-mounted servers are removed from the “MCP gateway config”, but the code now explicitly keeps them in the gateway config (so containers start) and only removes them from the agent’s final config in convert_gateway_config_*.sh. Please update the schema description so it matches the runtime behavior.
```suggestion
          "description": "When true, each user-facing MCP server is mounted as a standalone CLI tool on PATH. The agent can then call MCP servers via shell commands (e.g. 'github issue_read --method get ...'). CLI-mounted servers remain in the MCP gateway config so their containers can start, and are removed only from the agent's final config during convert_gateway_config_*.sh processing. Default: false.",
```
</comment_old>

</comment_thread>

<comment_thread>
<file>pkg/workflow/mcp_cli_mount.go:17</file>
<original_commit>d22ea30</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
The file header comment says CLI wrapper scripts are written to `/tmp/gh-aw/mcp-cli/bin/`, but mount_mcp_as_cli.cjs generates them under `${RUNNER_TEMP}/gh-aw/mcp-cli/bin` so they’re accessible inside the AWF sandbox. Please update this comment to match the actual output path.

This issue also appears on line 50 of the same file.
```suggestion
//   - Writes a standalone CLI wrapper script for each server to ${RUNNER_TEMP}/gh-aw/mcp-cli/bin/
```
</comment_old>

</comment_thread>

<comment_thread>
<file>pkg/workflow/mcp_cli_mount.go:24-28</file>
<original_commit>d22ea30</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>

internalMCPServerNames includes "mcp-scripts", but the actual MCP server ID used in rendered gateway configs is "mcpscripts" (see pkg/constants/job_constants.go) and the manifest will therefore contain "mcpscripts". As-is, enabling mount-as-clis can accidentally expose the internal mcpscripts server as a CLI. Consider excluding both "mcp-scripts" (config key) and "mcpscripts" (server ID) consistently.
```suggestion
// should not be exposed as user-facing CLI tools.
// Include both config-key and rendered server-ID variants where they differ.
var internalMCPServerNames = map[string]bool{
        "safeoutputs": true,
        "mcp-scripts": true,
        "mcpscripts":  true,
```
</comment_old>

</comment_thread>

<comment_thread>
<file>actions/setup/js/mount_mcp_as_cli.cjs:35</file>
<original_commit>d22ea30</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
INTERNAL_SERVERS excludes "mcp-scripts", but the gateway/server ID is "mcpscripts" (no hyphen). The CLI manifest will therefore contain "mcpscripts" and this step will mount it as a user-facing CLI unless it’s added to the internal exclusion list (or otherwise filtered).

This issue also appears on line 399 of the same file.
```suggestion
const INTERNAL_SERVERS = new Set(["safeoutputs", "mcp-scripts", "mcpscripts"]);
```
</comment_old>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_old>
<author>@pelikhan</author>

```
@copilot read comments
</comment_old>

<comment_old>
<author>@copilot</author>

&gt; @copilot read comments

Addressed all 5 review comments in 2946954:

1. Fixed `MountAsCLIs` doc comment in `tools_types.go` — now
correctly says servers stay in the gateway config and are
filtered from the agent's final config.
2. Updated `mount-as-clis` schema description to match the
actual runtime behavior (gateway config unchanged, agent config
filtered by `convert_gateway_config_*.sh`).
3. Fixed file header comment in `mcp_cli_mount.go` — output path
is now `${RUNNER_TEMP}/gh-aw/mcp-cli/bin/` (not `/tmp/gh-aw/`).
4. Added `"mcpscripts": true` to `internalMCPServerNames` in
`mcp_cli_mount.go` to exclude the rendered server ID.
5. Added `"mcpscripts"` to `INTERNAL_SERVERS` Set in
`mount_mcp_as_cli.cjs` for the same reason.

&gt; [!WARNING]
&gt;
&gt;</comment_old>

<comment_old>
<author>@pelikhan</author>
@copilot the generate bash script must have the gateway api key
to able to access the gateway endpoint
</comment_old>

<comment_old>
<author>@copilot</author>

&gt; @copilot the generate bash script must have the gateway api
key to able to access the gateway endpoint

Fixed in 85f06a0. The API key is now baked directly into the
generated bash wrapper scripts at generation time (when
`mount_mcp_as_cli.cjs` runs, before AWF starts), rather than
read from the environment at runtime. This is necessary because
`MCP_GATEWAY_API_KEY` is explicitly excluded from the AWF
sandbox via `--exclude-env MCP_GATEWAY_API_KEY`.
</comment_old>
```

```
<comment_old>
<author>@pelikhan</author>
@copilot merge main and recompile
</comment_old>

<comment_old>
<author>@copilot</author>

&gt; @copilot merge main and recompile

Done in 3a7b43c. Merged main and recompiled all 181 workflows
successfully.

&gt; [!WARNING]
&gt;
&gt;


</comment_old>

<comment_old>
<author>@pelikhan</author>
@copilot   **Confirmed Bug**: Generated bash wrapper scripts
(from `mount_mcp_as_cli.cjs:335`) have a syntax error — nested
double-quote escaping in the `jq -r` expression causes `bash:
unexpected EOF while looking for matching '"'`. Also flagged
`"agentic-workflows"` vs `"agenticworkflows"` name mismatch in
`mcp_cli_mount.go:54`.
</comment_old>

<comment_old>
<author>@copilot</author>

&gt; @copilot   **Confirmed Bug**: Generated bash wrapper
scripts (from `mount_mcp_as_cli.cjs:335`) have a syntax error —
nested double-quote escaping in the `jq -r` expression causes
`bash: unexpected E...

Fixed in 7526d82:

1. **Bash syntax error** (`mount_mcp_as_cli.cjs:335`): Switched
the multi-line `jq -r` expression from a double-quoted to a
single-quoted bash string. In the previous double-quoted form,
the `\"` at the end of the `elif` line was followed by a bare
`"` that prematurely closed the outer bash string, leaving
`else ... end"` dangling. Single-quoted bash strings need no
escaping for `"` characters.
```

```
2. **Name mismatch** (`mcp_cli_mount.go:54`):
`getMCPCLIServerNames` now emits `"agenticworkflows"` (the
gateway/manifest server ID) instead of `"agentic-workflows"`
(the frontmatter key). This ensures `GH_AW_MCP_CLI_SERVERS`
correctly matches the manifest entry names and the
`convert_gateway_config_*.sh` filter (`select(.key |
IN($cliServers[]) | not)`) actually excludes the server from the
agent's final config.

> [!WARNING]
>
> <details>
> <summary>Firewall rules blocked me from connecting to one or
more addresses (expand for details)</summary>
>
> #### I tried to connect to the following addresses, but was
blocked by firewall rules:
>
> - `https://api.github.com/graphql`
>   - Triggering command: `/usr/bin/gh /usr/bin/gh api graphql
-f query=query($owner: String!, $name: String!) {
                repository(owner: $owner, name: $name) {
                        hasDiscussionsEnabled
                }
        } -f owner=github -f name=gh-aw` (http block)
>   - Triggering command: `/usr/bin/gh /usr/bin/gh api graphql
-f query=query($owner: String!, $name: String!) {
                repository(owner: $owner, name: $name) {
                        hasDiscussionsEnabled
                }
        } -f owner=github -f name=gh-aw cidrPko/A8NKLbGP-C`
(http block)
>   - Triggering command: `/usr/bin/gh /usr/bin/gh api graphql
-f query=query($owner: String!, $name: String!) {
                repository(owner: $owner, name: $name) {
                        hasDiscussionsEnabled
                }
        } -f owner=github -...
</comment_old>

<comment_old>
<author>@pelikhan</author>
@copilot merge main and recompile
</comment_old>

<comment_old>
```

<author>@copilot</author>

> @copilot merge main and recompile

Done in efc9b05. Merged main and recompiled all 180 workflows successfully.

> [!WARNING]
>
></comment_old>

<comment_old>
<author>@pelikhan</author>
@copilot the agent is quite confused about the CLI the tools that CLI provide a better built-in prompts and update the instructions to smoke a pallet to avoid confusion about where to find those tools

See this run for more information look at the agent log https://github.com/github/gh-aw/actions/runs/23991069731/job/69970834693#step:45:1
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot the agent is quite confused about the CLI the tools that CLI provide a better built-in prompts and update the instructions to smoke a pallet to avoid confusion about where to find those tools...

Done in 9698016. Two improvements:

1. **`mcp_cli_tools_prompt.md`** — rewrote the prompt to make it unambiguous that the listed servers are **not MCP tools**. Added a bold IMPORTANT callout, concrete `github` and `playwright` examples with actual command syntax, and replaced the confusing "gateway" wording with "bash executables".

2. **`smoke-copilot.md`** — added a "Tool Access Overview" section at the top of the test instructions listing all four CLI-mounted servers and explicitly stating they must be used via the `bash` tool. Updated each relevant test step to say "use bash to run `<cli> <tool>`" to eliminate ambiguity.

> [!WARNING]
>

```
&gt;</comment_old>

<comment_old>
<author>@pelikhan</author>
@copilot Merge main Enrico recompile you Dumbo is it only react
to your voice
</comment_old>

<comment_old>
<author>@copilot</author>

&gt; @copilot Merge main Enrico recompile you Dumbo is it only
react to your voice

Done in 35bf4d2. Merged main and recompiled all 180 workflows
successfully.

&gt; [!WARNING]
&gt;
&gt;</comment_old>

<comment_old>
<author>@pelikhan</author>
See errors with mcpscripts https://github.com/github/gh-aw/
actions/runs/23992495407/job/69974540910#step:36:1
</comment_old>

<comment_old>
<author>@pelikhan</author>
@copilot investigate mcpscripts errors https://github.com/
github/gh-aw/actions/runs/23992495407/job/69974540910#step:36:1
</comment_old>

<comment_old>
<author>@copilot</author>

&gt; @copilot investigate mcpscripts errors https://github.com/
github/gh-aw/actions/runs/23992495407/job/69974540910#step:36:1

Fixed in acd44a6.

**Root cause:** `mount_mcp_as_cli.cjs` was baking raw gateway-
output URLs (e.g., `http://0.0.0.0:80/mcp/github`) into CLI
wrapper scripts. Those URLs work from the host where the mount
step runs, but fail inside the AWF sandbox container where the
gateway is only reachable via `host.docker.internal:80`.
```

Fix:
1. `start_mcp_gateway.sh` — added `gateway-domain` to step outputs
2. `mcp_cli_mount.go` — mount step now receives `MCP_GATEWAY_DOMAIN` and `MCP_GATEWAY_PORT` from gateway outputs
3. `mount_mcp_as_cli.cjs` — added `toContainerUrl()` that rewrites the raw manifest URL to use `MCP_GATEWAY_DOMAIN:MCP_GATEWAY_PORT` when generating CLI wrappers; tool fetching still uses the raw host-accessible URL; warns if the env vars are absent

> [!WARNING]
>
></comment_old>

<comment_old>
<author>@pelikhan</author>
@copilot the agent is still very co fused about how to call the clis

https://github.com/github/gh-aw/actions/runs/23993205511/job/69976454416
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot investigate mcpscripts errors https://github.com/github/gh-aw/actions/runs/23992495407/job/69974540910#step:36:1

Fixed in 3214e78. Root cause: `export GH_AW_MCP_CLI_SERVERS=...` in the start-mcp-gateway step only set the variable for that step's shell session. The `convert_gateway_config_*.sh` scripts run in a subsequent step and saw an empty variable (defaulting to `[]`), so no servers were filtered from the agent's final MCP config — leaving the agent with both direct MCP access AND CLI wrappers simultaneously, causing the confusion.

Fix: added `echo 'GH_AW_MCP_CLI_SERVERS=...' >> "$GITHUB_ENV"` in `mcp_setup_generator.go` so the variable persists to subsequent steps via the GitHub Actions environment file mechanism.

> [!WARNING]
>
></comment_old>

```
<comment_old>
<author>@pelikhan</author>
@copilot merge main and recompile
</comment_old>

<comment_old>
<author>@copilot</author>

&gt; @copilot merge main and recompile

Done in 81db7fa. Merged main and recompiled all 181 workflows
successfully.

&gt; [!WARNING]
&gt;
&gt;


</comment_old>
  <comment_new>
  <comment_id>4190112072</comment_id>
  <author>@pelikhan</author>
  @copilot merge main and recompile
  </comment_new>

</pr_comments>

</comments>

----
The last **24** git commits in this branch are the changes you
have made so far. Use those as your change commit history.
</problem_statement>


Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address the
<comment_new> comments. Reply to new comments as you address them.
Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
    - Is it a general comment or praise.
```

* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.